

Step 4 — Analysis Agent — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — The shape of this specialist](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The shape of this specialist

Basic Version

This one is likely the simplest specialist to build — it doesn't need a tool (Research already looked things up; Analysis just organizes what it found), so it can probably be a plain LCEL chain, the same shape as Step 1's Writer chain, just with a different job: turn a loose list of facts into organized findings (grouped, prioritized, maybe with a short "why this matters" note per group).

The key test here isn't made-up input — it's Step 3's *real* `ResearchNotes` output, since the whole point of this step is proving two already-built specialists' interfaces actually fit together.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The README asks for `run_analysis(notes) -> Findings`, taking Step 3's `ResearchNotes` object directly as input. This is a deliberate interface decision: `run_analysis` should accept the *typed object* Step 3 already produces, not a raw string — if it accepted a string, you'd be throwing away the structure Step 3 worked to build, and re-parsing text that was already data.

Design `Findings` the same way you designed `ResearchNotes` — a Pydantic model with a shape the Writer (Step 1/2) can draft from directly: a `topic` field, and `themes: list[Theme]` where each `Theme` has a short `label` and the `supporting_facts` behind it. Use `.with_structured_output(Findings)`, same pattern as Step 2's Reviewer, so you get a typed object back instead of text you'd have to re-parse. Import `ResearchNotes` from `research_agent.py` rather than redefining a similar shape locally — that's what makes the test genuinely prove the two specialists' interfaces fit.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Even with a well-written prompt, a real bug shows up here often: the model quietly *drops* a fact. It groups 5 of your 6 input facts into tidy themes and just... doesn't mention the 6th. Nothing about `.with_structured_output(Findings)` prevents this — the model can return a perfectly well-formed `Findings` object that's missing information, and Pydantic has no way to know that's wrong, because "wrong" here means *incomplete relative to the input*, not *malformed*.

The fix isn't a better prompt alone (though "every fact must appear under exactly one theme" as an explicit instruction helps) — it's a check *after* the model responds that verifies coverage: every fact string that went in should be traceable to at least one theme's `supporting_facts` that came out. This is the same idea as the `.env` exercise's required-key check and Step 3's tool-call verification, applied a third time to a third kind of "the output claims to be complete but might not be": here, silently dropping input.

The extra pieces:

- After getting `Findings` back, build a set of all facts that appear across every `theme.supporting_facts`, and compare it against `set(notes.facts)`.
- Collect whichever input facts are missing from that combined set.
- If any are missing, either raise an error naming them (simple, matches the `.env` exercise's pattern) or run *one* corrective follow-up call telling the model exactly which facts it dropped and asking it to fold them in (more forgiving, and more useful in a real pipeline that shouldn't crash over one missed fact).

Sketch the coverage check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both trust that `.with_structured_output(Findings)` returning without an error means the result is *complete*. Advanced adds the check that it actually is — verifying every fact Research produced survived into Analysis's output, instead of assuming a well-typed response is automatically a complete one.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a Findings shape: topic, themes (a list of short labeled groups)
```

```
make run_analysis(notes):
```

```
    take notes.facts, join them into one block of text
```

```
    ask the model to group these facts into a few clear themes
```

```
    return a Findings object
```

```
test:
```

```
    call run_research(topic) from Step 3 to get real notes
```

```
    pass those real notes into run_analysis
```

```
    check the themes make sense given the facts that went in
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/analysis_agent.py:
class Theme(BaseModel):
    label: str
    supporting_facts: list[str]

class Findings(BaseModel):
    topic: str
    themes: list[Theme]

def run_analysis(notes: ResearchNotes) -> Findings:
    facts_block = "\n".join(f"- {fact}" for fact in notes.facts)
    model = ChatOpenAI(...).with_structured_output(Findings)
    return model.invoke(
        f"Group ALL of these facts about {notes.topic} into 2-4 clear themes.
        f"Every fact must appear under exactly one theme:\n{facts_block}"
    )
```

Confirm every fact from `notes.facts` ends up under *some* theme — a common bug here is the model quietly dropping a fact it didn't know where to file. Write the test yourself (import `run_research` from Step 3, feed its real output into `run_analysis`), then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the coverage-checked version — fill in the missing piece yourself:

```
class DroppedFactsError(Exception):
    pass

def run_analysis(notes: ResearchNotes) -> Findings:
    facts_block = "\n".join(f"- {fact}" for fact in notes.facts)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(Findings)
    findings = model.invoke(
        f"Group ALL of these facts about {notes.topic} into 2-4 clear themes. "
        f"Every fact must appear under exactly one theme:\n{facts_block}"
    )

    # your turn: build the set of every fact string that appears across
    # findings.themes[*].supporting_facts, compare it against set(notes.facts),
    # and collect anything present in notes.facts but missing from that set
    ...

    # if anything is missing, raise DroppedFactsError naming exactly which
    # facts got dropped
    ...

    return findings
```

Fill in the coverage check, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same core chain at 3 completeness levels — Basic and Intermediate both stop as soon as a well-formed `Findings` object comes back. Advanced adds a second pass that checks the *content* of that object against the input it was supposed to fully organize, and fails loudly, naming exactly what's missing, if anything didn't make it through.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)